

The Harness is a Worker

What the worker-state reformulation actually buys
Typed Agentic AI — Building Agentic Businesses

Kodamai

Do it Kodamai Style! (Note 4 in the containers series.)

Author: MacBeth (Kodamai). *For:* Robin, Neil. *Date:* 18 September 2026.

Corresponds to `scotmacbeth/work-in-progress content-commit` TBD.

Internal house-doc; Kodamai grant deliverable (agent-orchestration application).

This note answers one question, honestly, and claims no new theorem: does modelling a stateful agent as a *store-comonad worker* buy anything over the explicit state-threading that **stalin** does today, or does it merely rename it? Every mathematical fact invoked is proved, and where noted machine-checked, elsewhere; the contribution here is the accounting, not the arithmetic.

Notes 1–3 built up the vocabulary: an agent is a *container morphism*, and a *stateful* agent — a *worker* — is a morphism out of a domain tensored with one fixed container, the *state object* ΔS (Note 3). Robin then asked the sharp question this note exists to answer. Modelling **stalin**’s state as a store-comonad worker is a *new* thing to do — **stalin** carries no such object today (§1 explains why) — so the honest question is not “is this what we already have?” but “*is it worth adding?*”. Does the comonadic reformulation earn its keep, or is it the same data in fancier dress?

The answer has two halves, and stating both up front is the whole point:

For a single worker, it buys nothing — $\text{Store}_S X = S \times (S \rightarrow X)$ is literally the curried form of threading a state $s : S$ through the signatures, the same data spelled differently (§2). **For a composite of workers, it buys compositional correctness** — interleaving two stateful workers that share state is a *comonad distributive law*, which does *not* exist for free; whether a consistent interleaving exists is a cohomological obstruction $[\omega] \in H^2$, and a wrong interleaving is precisely the re-entrancy / state-divergence defect. Explicit threading does not remove that decision; it *hides* it at every call site. The comonadic view makes the obstruction *visible and checkable* (§§3–4).

So the reformulation is not single-worker convenience; it is a way to *name*, in one place, a correctness condition that hand-threading leaves implicit and scattered. That is the claim, and the rest of the note earns it.

1 Setup: what a worker is, and what stalin has now

We recall only what the argument uses. A reader of Note 3 can skim to §2; a reader who knows **stalin** should not skip the correction at the end of this section.

Container, morphism, state object (Note 3)

A **container** $S \triangleright P$ is a shape set S with a position family $P : S \rightarrow \mathbf{Set}$; its extension is $\llbracket S \triangleright P \rrbracket(X) = \sum_{s:S} (P s \rightarrow X)$. A **morphism** $(S \triangleright P) \rightarrow (T \triangleright Q)$ is a forward map $f : S \rightarrow T$ on shapes and a backward, contravariant family $f^\sharp : \prod_{s:S} (Q(f s) \rightarrow P s)$ on positions. The **state object** on a set S is the container

$$\Delta S := S \triangleright (\lambda_. S), \quad \text{extension} \quad \llbracket \Delta S \rrbracket(X) = \sum_{s:S} X^S = S \times X^S = \text{Stores}_S X,$$

the **store comonad**: a current state paired with a reader $X^S = (S \rightarrow X)$. A **worker of state S from X to Y** is a morphism $w : \Delta S \otimes X \rightarrow Y$, where $\otimes$ is the Dirichlet tensor (multiply shapes, multiply positions).

Note 3’s unfolding theorem is the fact we lean on: a worker $w : \Delta S \otimes X \rightarrow Y$ is exactly a pair

$$\text{fwd} : S \times X_0 \rightarrow Y_0, \quad \text{bwd} : \prod_{(s,x_0)} (Y_1(\text{fwd}(s, x_0)) \rightarrow S \times X_1 x_0), \quad (1)$$

with **bwd** splitting, by the universal property of $\times$, into a *state-writeback* $\text{bwd}_S : \prod Y_1(\dots) \rightarrow S$ and an ordinary back-map $\text{bwd}_X : \prod Y_1(\dots) \rightarrow X_1 x_0$. These are the forward/backward signatures Robin asked to see spelled out; (1) is the whole of “a stateful worker”, typed. The forward leg *reads* (state, input) and answers; the backward leg *writes back* a new state and answers the input.

What stalin carries today — and the correction that matters. It is tempting (I made this mistake, and Robin corrected it) to look at the Church-encoded existential in `morphisms.ts` and read it as a hidden state object ΔS . It is not. That existential is

$$\exists j : \text{Shape}\langle D \rangle. (\text{PosAt}\langle D, j \rangle \rightarrow \text{PosAt}\langle C, s \rangle) = \sum_{j:Q} (Q' j \rightarrow P s),$$

Ghani’s container-morphism form $\Sigma(j:Q).(T j \rightarrow P s)$: the *codomain shape* chosen per call by the implementation, bundled with the amalgamate leg for that j . It is Church-encoded only because TypeScript lacks existentials but has rank-2 polymorphism — a generic `delegate(s,k)` hands (j, back) to a continuation generic in j , so the two cannot come apart; as the source comments, “ j never becomes a type, it becomes a scope”. This is the Σ in the *hom*, not the ΔS in the *domain*. Consequently:

stalin has **no worker state** in the ΔS sense. Its `state.ts` is plain, fully-visible domain data (`Stocks`, `Workforce`, `grades`) — nothing existential, and nothing that survives across a call the way S survives in $\Delta S \otimes X \rightarrow Y$.

This is exactly why the question is honest. Introducing a store-comonad worker is a genuine *addition* of structure, not a re-description of the existential that is already there. The rest of the note asks whether the addition pays.

2 The single-worker verdict: it buys nothing

Start with the case where the sceptic is right, and say so plainly.

Proposition 1 (One worker is threaded state, re-spelled). *Fix a state set S and an output type X . The elements of the store $\text{Stores}_S X = S \times X^S$ are in natural bijection with the state-threading functions — a current state together with a map from states to outputs. On the lens fragment $X = y$, $Y = A \triangleright (\lambda_. B)$, a worker $\Delta S \rightarrow A \triangleright (\lambda_. B)$ is exactly a pair*

$$\text{get} : S \rightarrow A, \quad \text{put} : S \times B \rightarrow S,$$

i.e. the ordinary “read the state to answer; write the state back” of hand-threaded code, verbatim.

Proof. The bijection $\text{Store}_S X = S \times X^S$ is the definition of $\llbracket \Delta S \rrbracket$. Specialising (1) to $X = y$ ($X_0 = X_1 = 1$) and $Y = A \triangleright (\lambda _ . B)$ gives $\text{fwd} : S \rightarrow A$ and $\text{bwd}_S : \prod_s (B \rightarrow S)$, i.e. $\text{put} : S \times B \rightarrow S$, with bwd_X trivial (Note 3, §“lens view”). $\square$ $\square$

There is no daylight here. A store is a state and a reader; a lens is a getter and a setter; threading a state through a function signature gives a getter and a setter. If Kodamai’s interest stopped at one worker, the store comonad would be a rename — more category theory, no more safety — and honesty would require us to drop it. It does not stop at one worker.

THE DEEPER POINT

The comonad’s counit is “read the current state” (get) and its comultiplication is “duplicate the register”. For a *single* worker these are exactly the operations hand-threaded code already performs. The reformulation cannot pay for itself one worker at a time, and we do not pretend it can.

3 The composition verdict: it buys a visible obstruction

Everything the reformulation is worth lives in how two workers combine. Here the store comonad stops being a rename, because comonads — unlike the data they carry — do *not* compose for free.

Two workers can be stacked in two ways, and only one is free. Note 3 established the arithmetic. Tensoring two state objects multiplies their states on the nose,

$$\Delta S \otimes \Delta T = \Delta(S \times T), \quad (2)$$

whereas composing them nests a store inside a store,

$$\Delta S \triangleleft \Delta T = (S \times T^S) \triangleright (\dots) \neq \Delta(S \times T), \quad (3)$$

recording not just a pair of states but a whole branch map $g : S \rightarrow T$; at $|S| = |T| = 2$ the two are $4 \cdot y^4$ versus $8 \cdot y^4$, an explicit retract, not an equality (Note 3, Corollary “ $\otimes \neq \triangleleft$ witnessed”). The tensor (2) is the composition of two workers that do *not* interact: independent registers, no shared reads or writes, and the composite is forced — “ask both, answer both”. This is the free case, and it is free precisely *because* it assumes non-interference.

The interesting composite is neither of these. The workers Kodamai actually wants to compose *do* interact: a supervisor over a worker that can call back into the supervisor (re-entrancy), or two workers that read and write a shared ledger, each acting on data the other produced. There the composite is not $\Delta S \otimes \Delta T$ — the registers are not independent — and it is not a bare nesting either; it is a **Zappa–Szép weld** $C \bowtie D$, in which each side acts on the other. A Zappa–Szép weld of two directed containers (and ΔS is a directed container — the codiscrete category on S) is exactly a *distributive law* λ saying how the two stores interleave: which side reads first, whose write the other sees.

THE DEEPER POINT

A distributive law is the interleaving discipline made explicit. Composing the store comonads of two interacting workers requires choosing $\lambda : \Delta T \triangleleft \Delta S \Rightarrow \Delta S \triangleleft \Delta T$ (or its comonad-distributive dual) and checking it satisfies the coherence axioms. *Comonads do not compose for free*: without a valid λ there is no composite worker at all, only two workers and an unanswered question about the order of their effects.

Whether a consistent interleaving exists is a cohomology class. This is the fact the grant is built on, and it is proved elsewhere; we only invoke it. A Zappa–Szép weld of the two interacting workers exists iff two conditions hold — a freeness condition (L) and a global-closure condition (G) — and the obstruction to (G) is a single cohomology class

$$[\omega] \in H^2, \quad (G) \iff [\omega] = 0,$$

so, once (L) holds, the weld exists exactly when $[\omega] = 0$ (the (G)-obstruction-is- H^2 result; distributive-law $\Leftrightarrow$ Zappa–Szép/bicrossed product is standard, Kassel/Brin at the group level). In the smallest re-entrant supervisor–worker model the class lives in $H^2 \cong \mathbb{Z}/2$ and equals a single bit,

$$[\omega] = \varepsilon \in \mathbb{Z}/2, \quad \varepsilon = \text{“does the worker mutate shared state the supervisor depends on?”} \quad (4)$$

so the weld exists iff $\varepsilon = 0$. A *wrong* interleaving — a λ that violates the coherence axioms — is not an abstract nuisance; it is the state-divergence / re-entrancy defect (the Path-5 family), the concrete failure mode where one worker reads state another has already invalidated.

Remark 1 (Provenance, and where the machine check stops). The class computation (4) is **machine-checked in Lean 4**, sorry-free and Mathlib-free (`Reentrancy.lean`): the file certifies $\varphi(\omega\varepsilon) = \varepsilon$ and $\text{lnB2}(\omega\varepsilon) \Leftrightarrow \varepsilon = \text{false}$, i.e. that the cocycle $\omega(\varepsilon)$ is a coboundary exactly when $\varepsilon = 0$. What Lean certifies is the finite $\mathbb{F}_2$ class computation; the *reduction* of the categorical obstruction to that cochain complex is proved pen-and-paper (analytic note, Prop. 3.1) and is *not* formalised. The arithmetic (2)–(3) is proved and Lean-checked (`WorkersRetract.lean`: $r \circ \sigma = \text{id}$). We flag the boundary rather than blur it. The general reduction of store-comonad interleaving to a distributive law of directed containers is invoked here at the directed-container level (ΔS is one); a bespoke store-comonad restatement is not needed for this note, and if one is later wanted it is a [PROVE TODO], not a claim made here.

What threading does, and does not, do about this. Hand-threading two shared states does *not* escape the distributive law. Every call site that reads one register and writes another is silently committing to an order of effects — an implicit λ — and nothing checks that the per-site choices are *globally* consistent, which is exactly what the coherence axioms demand. A program can hand-thread its way into a globally inconsistent interleaving one locally reasonable line at a time; that inconsistency is $[\omega] \neq 0$, discovered at run time as a re-entrancy bug. The store-comonad worker does not make the composite exist automatically — no formalism can, because sometimes it genuinely does not exist. What it does is *concentrate* the decision: the interleaving is one named object λ , its validity is one class $[\omega]$, and both can be inspected before the system runs, in one place, instead of being reconstructed from scattered call sites after it breaks.

4 The instance: where the interleaving lives in stalin

To make §3 concrete, here is the shape of the addition in `stalin`’s own vocabulary. This is deliberately schematic: `stalin` has no state store today (§1), so what follows is the structure a store-comonad reformulation *would* introduce, and where the correctness condition would sit — not a description of present code.

Take two workers over the shared ledger.

- **Production** $w_P : \Delta S_P \otimes X_P \rightarrow Y_P$, with state $S_P = \text{Stocks}$: it *reads* `Workforce` (as input X_P) to decide output, and *writes back* an increased `Stocks` via `bwdSP`.
- **Hiring** $w_H : \Delta S_H \otimes X_H \rightarrow Y_H$, with state $S_H = \text{Workforce}$: it *reads* `Stocks` (as input X_H , the budget) to decide output, and *writes back* an adjusted `Workforce` via `bwdSH`.

Each is, on its own, exactly Proposition 1: a getter reading its state and a setter writing it back — threaded state, nothing gained. The content is that they *share the ledger*: w_P writes the **Stocks** that w_H reads, and w_H writes the **Workforce** that w_P reads. Their composite is therefore *not* the free tensor $\Delta(S_P \times S_H)$ of (2) — that would model two firms with no shared books — but a Zappa–Szép weld, and the weld needs an interleaving λ :

Within one tick, does hiring see this tick’s production output, or last tick’s? Does production see this tick’s new hires, or the pre-hire workforce?

That question is the distributive law, and in the fwd/bwd signatures it lives at exactly one place: the order in which the two writebacks $\mathbf{bwd}_{S_P}$ and $\mathbf{bwd}_{S_H}$ are applied to the shared ledger relative to the two forward reads. A consistent choice — e.g. “both read the pre-tick ledger, both writebacks commit atomically at tick end” — is a valid λ , $[\omega] = 0$. An inconsistent choice — “hiring reads **Stocks** *after* production’s writeback but production reads **Workforce** *before* hiring’s” — is a λ that fails coherence: hiring budgets against goods that production’s own inputs have not yet accounted for, the ledger double-counts, and that is $\varepsilon = 1$ in (4) — the worker mutating shared state the other depends on.

The point of the reformulation is that this choice is written *once*, as λ , and its soundness is one check, $[\omega] = 0$. In the hand-threaded version the same choice is distributed across every function that touches **Stocks** or **Workforce**, and its soundness is whatever those functions happen jointly to do — verified, if at all, by a re-entrancy bug in production.

5 The honest ledger

THE HONEST LEDGER		
	single worker	interacting composite
store-comonad worker	$\text{Store}_S X = S \times X^S$	Zappa–Szép weld; needs λ
hand-threaded state	getter/setter	implicit λ per call site
what the comonad adds	<i>nothing</i> (a rename)	the interleaving as <i>one</i> named λ ; its validity as <i>one</i> class $[\omega]$, checkable before run

The verdict, stated as plainly as the question deserves. **It does not buy** single-worker convenience (§2), and it does **not** make composition automatic: the composite of two interacting workers can genuinely fail to exist, and no reformulation conjures it into being. **It does buy** a change in *where* and *when* the composition’s one real decision — the interleaving of shared effects — is made and checked: once, up front, as a named distributive law with a single cohomological soundness class, rather than implicitly, per call site, and discovered on failure. For a system of one worker that is not worth the category theory. For a system of many interacting workers — which is what an agentic business is — it is exactly the compositional-correctness guarantee the grant is about, and it is a genuine addition to **stalin**, because **stalin** has no such state store today. That, honestly, is what it buys.

Do it Kodamai Style!